Fundamentals of Cybersecurity

Mohammad Hosseini

Faculty of Computer Science and Engineering
Shahid Beheshti University

Programs and Programming



Introduction

In this chapter we describe security problems in **programs** and **programming**, with an eye toward understanding and recognizing various types of software failures.

Some security problems result from nonmalicious oversights but others are intentional.

- A malicious attacker can exploit a nonmalicious flaw to cause real harm.
- We study several common program failings to show how simple errors during programming can lead to large-scale problems during execution.

Part 1:

Unintentional (nonmalicious) programming oversights

Program flaws

Program flaws can have two kinds of security implications:

- Causing integrity problems leading to harmful output or action
- Offering an opportunity for exploitation by a malicious actor

Some programming flaws that have security implications

- Buffer overflow
- Incomplete mediation
- Off-by-one error
- Unsafe utility program
- Time-of-check to time-of-use
- Undocumented access point
- Integer overflow
- Unterminated null-terminated string

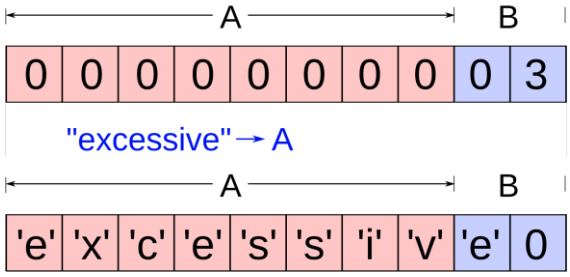
Buffer Overflow

Buffer overflow

- A buffer is a space in memory in which data can be held.
- A buffer overflow occurs when a program writes data to a buffer beyond the buffer's allocated memory.
  - As a result, the program overwrites adjacent memory locations



Buffer overflow



- The impact of an overflow can be disproportionate to the underlying oversight.
  - They can be used for malicious ends.

Buffer overflow attack

By sending in data designed to cause a buffer overflow,

- it is possible to write into areas known to hold executable code and replace it with malicious code,
- or to selectively overwrite data pertaining to the program's state,
- causing behavior that was not intended by the original programmer.

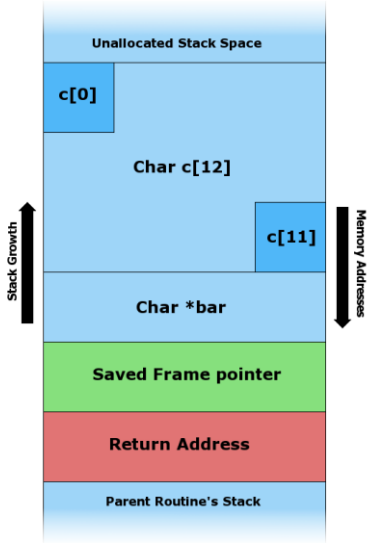
Buffer overflow attack example: stack overflow

```c
#include <string.h>

void foo(char *bar)
{
   char c[12];

   strcpy(c, bar);  // no bounds checking
}

int main(int argc, char **argv)
{
   foo(argv[1]);
   return 0;
}
```



Stack

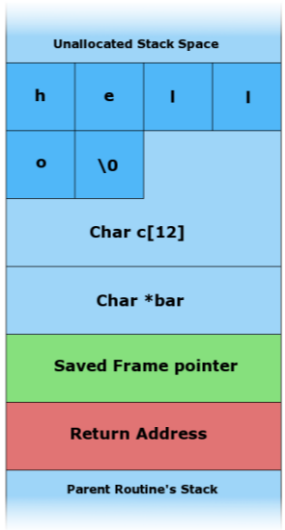
Buffer overflow attack example: stack overflow

```c
#include <string.h>

void foo(char *bar)
{
   char c[12];

   strcpy(c, bar);  // no bounds checking
}

int main(int argc, char **argv)
{
   foo(argv[1]);
   return 0;
}
```



Command line argument:
"hello"

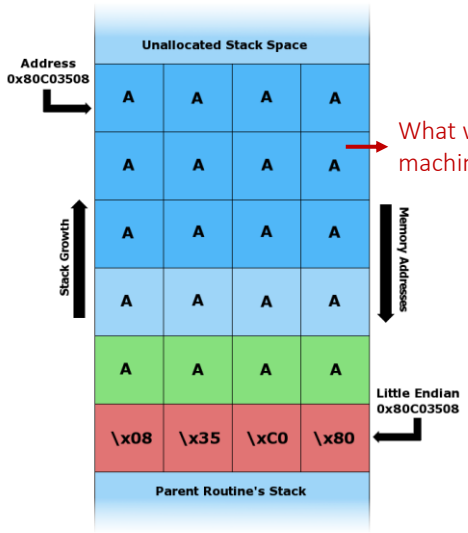
Buffer overflow attack example: stack overflow

```
#include <string.h>

void foo(char *bar)
{
   char c[12];

   strcpy(c, bar);  // no bounds chec
}

int main(int argc, char **argv)
{
   foo(argv[1]);
   return 0;
}
```

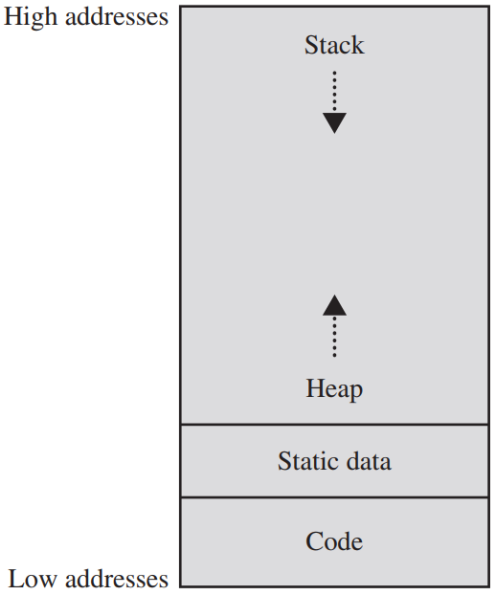


What would happen if these were machine-executable codes?

Command line argument:
"AAAAAAAAAAAAAAAAAAAA
A\x08\x35\xC0\x80"

Solution?

Heap overrun

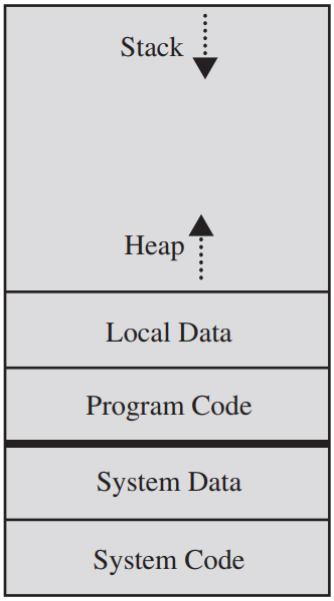


The stack and heap grow toward each other, and at some point they might collide.

The attacker wants to overwrite stack, sometimes called **stack smashing**, in a purposeful manner:

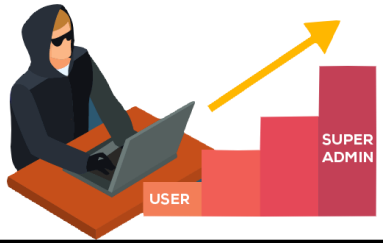
- **Overwrite the program counter** stored in the stack so that when this routine exits, control transfers to the address pointed at by the modified program counter address.
- **Overwrite the program counter and data** in the stack so that the program counter now points into the stack, causing the data overwritten into the stack to be executed.

Privilege escalation



- Every program is invoked by an operating system that may run with higher privileges than those of a regular program.
- By replacing a few instructions right after returning from his or her own procedure, the attacker regains control from the operating system, possibly with raised privileges.
- If the buffer overflows into system code space, the attacker merely inserts overflow data that correspond to the machine code for instructions.

Privilege escalation



Buffer overflow by parameter passing

An alternative style of buffer overflow occurs when parameter values are passed into a routine

- Passing parameters to a web server

```
http://www.somesite.com/subpage/userinput.asp?parm1=(808)555-1212
```

➢ What does the server do with a really long telephone number, say, one with 500 or 1,000 digits?

Buffer overflow in programming

```
char sample[10];
```
➢ Declaration

```
sample[10] = 'B';
```
➢ Compiler can detect the problem

```
sample[i] = 'B';
```
➢ During execution, the system can produce an error message.

Buffer overflow in programming

- In some languages, buffer sizes do not have to be predefined, so there is no way to detect an out-of-bounds error.
- The code needed to check each subscript against its potential maximum value takes time and space during execution.
  (resources are applied to catch a problem that occurs relatively infrequently)
- Even if the compiler were careful in analyzing the buffer declaration and use, this same problem can be caused with pointers, for which there is no reasonable way to define a proper limit.

➢ Thus, some compilers do not generate the code to check for exceeding bounds.

Buffer overflow countermeasures

- Check lengths before writing.
- Confirm that array subscripts are within limits.
- Monitor input and accept only as many characters as can be handled.

Some programming languages have features that preclude overflows. For example, languages such as Java, .NET, Perl, and Python generate code to check bounds before storing data.

The unchecked languages C, C++, and assembler language allow largely unlimited program access.

Incomplete Mediation

Incomplete mediation

Verifying that the subject is authorized to perform the operation on an object is called mediation.

Attackers exploit incomplete mediation to cause security problems.

Example

```
http://www.somesite.com/subpage/userinput.asp?
parm1=(808)555-1212&parm2=2019Jan17
```

The client's (user's) web browser enters the two values in their specified format for easy processing on the server's side.

- What would happen if parm2 were submitted as `1800Jan01`? Or `2019Feb30`? Or `2048Min32`? Or `1Aardvark2Many`?
  - One possibility is that the system would fail catastrophically.
  - Another possibility is that the receiving program would continue to execute but would generate a very wrong result.

Bad input

- Programmers typically dismiss considering bad input, asking why anyone would enter such numbers.
- But, just because something is senseless, stupid, or wrong doe not prevent people from doing it.

➢ Programmers should not assume data will be proper; instead, programs should validate that all data values are reasonable before using them.

Validate all input

- For instance, the programmer in the previous example may have written code to check for correctness on the client's side (the user's browser).

  ➢ The client program can search for and screen out errors.

- Or, to prevent the use of nonsense data, the program can restrict choices to valid ones only.

  e.g., drop-down box or choice list from which only the 12 conventional months could have been selected.

Complete mediation

- The application (previous slide) is still vulnerable.
  - The user can edit the URL line, change any parameter values, and send the revised line.
  - The data values are not completely mediated.

❖ Solution: complete mediation

Time-of-Check to Time-of-Use

Time-of-check to time-of-use

- The time-of-check to time-of-use (TOCTOU) flaw concerns mediation that is performed with a "bait and switch" in the middle.
  - Exploiting the **delay** between the two actions: check and use.

❖ Between access check and use, data must be protected against change.

Example

- Suppose a request to access a file were presented as a data structure, with the name of the file and the mode of access presented in the structure.
- Once authorized, the ticket is put on a queue of things to be done.

File: my_file	Action: Change byte 4 to A



File: your_file	Action: Delete file

Between the time the access was checked (by mediator) and the time the result of the check was used, a change occurred, invalidating the result of the check.

Countermeasures

We must be wary whenever a time lag or loss of control occurs, making sure that there is no way to corrupt the check's results during that interval.

- The validation routine can initially **copy** data from the user's space to the routine's area—out of the user's reach—and perform validation checks on the copy.
- The validation routine can seal the request data to detect modification.

Another example

- In **Unix**, the following C code, when used in a **`setuid`** program, has a **TOCTOU** bug

```
1  if (access("file", W_OK) != 0) {
2      exit(1);
3  }
4
5  fd = open("file", O_WRONLY);
6  write(fd, buffer, sizeof(buffer));
```

`access` checks the **real userid** rather than **effective userid**

- After the access check, before the open, the attacker replaces file with a **symlink** to the Unix password file /etc/passwd

Undocumented Access Point

Undocumented access point

- During program development and testing, programmers need a way to access the internals of a module.
  - Programmers create undocumented entry points or execution modes.
- An undocumented access point is called a **backdoor** or trapdoor.
  - Such an entry can transfer control to any point with any privileges the programmer wanted
- Sometimes, programmers forget to remove these entry points when the program moves from development to product; or the programmer decides to leave them in to facilitate program maintenance later.

Other software flaws

Reading assignment

- Off-by-one error
- Integer overflow
- Unterminated null-terminated string
- Unsafe utility program

Part 2:

Malicious code

Malware

Malware: MALicious software

- Any software **intentionally** designed to cause disruption to a computer, leak private information, gain unauthorized access to information or systems, or deprive access to information.

Most popular forms:

- Virus
- Worm
- Trojan horse

Virus

Virus: a type of malware that, when executed, replicates itself by modifying other nonmalicious programs and inserting its own code into those programs.

- A virus generally requires a **host program**.
  - An infected program itself begins to act as a virus, infecting other programs.

Worm

Worm: a **standalone** malware that replicates itself in order to spread to other computers.

- It often uses a computer network to spread itself, relying on security failures on the target computer to access it.

Trojan horse

Trojan: program with benign apparent effect but second, hidden, malicious effect.

- Trojan horse is on the surface a useful program with extra, undocumented (malicious) features.
- It does not necessarily try to propagate.

Other types of malware

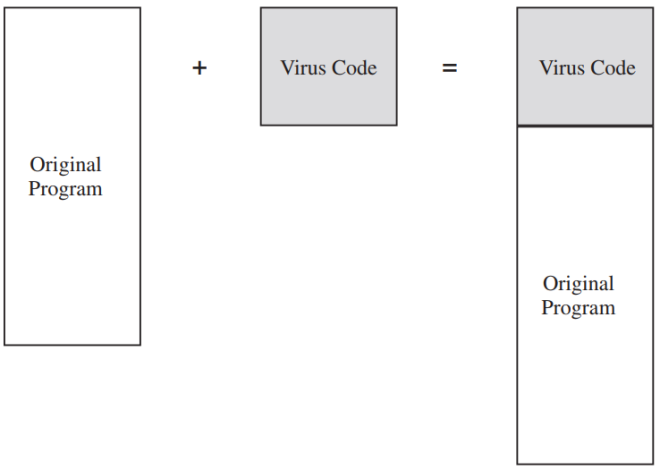
- Rabbit
- Logic bomb
- Time bomb
- Ransomware
- Spyware

Transmission and propagation of malware

- Setup and installer program
- Attached file (to a message)
- Document virus
- Autorun

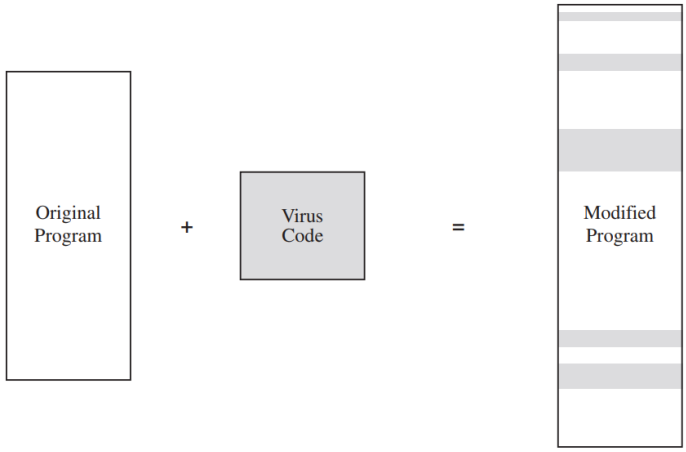
Transmission and propagation of malware

- Appended virus
- Easy to design and implement
- The virus writer need not know anything about the program to which the virus will attach, and often the attached program simply serves as a carrier for the virus.



Transmission and propagation of malware

- Integrated virus
- The virus writer has to know the exact structure of the original program to know where to insert which pieces of the virus.



Challenge of antivirus programs

- Detection tools are necessarily retrospective, looking for patterns of known infections.
- They try to find **virus-patterns** inside ordinary programs by scanning them for so-called virus signatures.
- As new infectious code types are developed, tools need to be updated frequently with new patterns.

➢ There will be infections that are too new to have been analyzed and included in the latest pattern file.

Challenge of antivirus programs

- Polymorphic viruses and worms modify their code on each infection.
- ➢ Many antivirus programs employ **heuristic analysis** to detect previously unknown computer viruses, as well as new variants of viruses already in the wild.

Countermeasures

- Use only commercial software acquired from reliable, well-established vendors.
- Test all new software on an isolated computer.
- Open attachments only when you know they are safe.
- Recognize that any website can be potentially harmful.
- Make a recoverable system image and store it safely.